Experiment 9: INTERFACES IN JAVA

Aim: To implement programs to demonstrate use of interfaces for multiple inheritance in java.

General Instructions (Not to be written in the file):

1. In every program, default and parametrized constructors have to be written in the classes.
2. Every class should have a toString() method that displays the details of the objects in a user friendly format. This method should be in addition to any display method mentioned in the problem statements.
3. All objects have to be created through user input.

Problem Definition: Write Java programs to:

BATCH A:

Q1. **Banking System with Multiple Account Types:** Design a banking system where Taxable and Auditable interfaces define default methods for tax computation and audit logging. A SavingsAccount class implements both. Taxable has a default method calculateTax(double balance) that returns 2.5% of balance above ₹1,00,000. Auditable has a default method generateAuditScore(int transactions) returning a risk score (transactions > 100 ? HIGH : LOW). SavingsAccount must override calculateTax() to use 1.8% instead, call both defaults, and produce a combined account health report with net balance after tax.

Q2. **Aerospace Vehicle Simulation System**

Inheritance Structure:

Propellable
└── AeroDynamic (extends Propellable)
    └── SupersonicCapable (extends AeroDynamic)

Trackable
└── MissionPlannable (extends Trackable)

Vehicle (base class)
└── Aircraft (extends Vehicle)
└── FighterJet (extends Aircraft)
implements SupersonicCapable, MissionPlannable

Interface Definitions:

- Propellable — abstract: engineThrust(double fuelFlow); default: fuelEfficiency(double thrust, double fuelBurn) → specific impulse = thrust / (fuelBurn * 9.81) in seconds
- AeroDynamic (extends Propellable) — abstract: liftCoefficient(double velocity, double wingArea); default: dragForce(double Cd, double airDensity, double velocity, double refArea) → 0.5 * Cd * airDensity * v^2 * A; default: liftToDragRatio(double lift, double drag)
- SupersonicCapable (extends AeroDynamic) — abstract: machNumber(double velocity, double speedOfSound); default: sonicBoom(double flightAltitude, double machNo) → overpressure = 0.53 * $machNo^2$ / sqrt(flightAltitude) in Pascals; default: heatGenerated(double machNo) → stagnation temperature = ambientTemp * (1 + 0.2 * $machNo^2$)
- Trackable — default: radarCrossSection(double length, double width) → RCS estimate = π * length * width / λ^2 (λ = 0.03m); abstract: currentPosition(double time)
- MissionPlannable (extends Trackable) — abstract: missionRange(double fuelCapacity, double burnRate); default: combatRadius(double totalRange) → totalRange / 2.5

Class Definitions:

- Vehicle — fields: vehicleId, mass, maxSpeed; method: kineticEnergy(double velocity) → 0.5 * mass * v^2

- Aircraft (extends Vehicle) — adds wingSpan, altitude, fuelCapacity; provides engineThrust() using afterburner model: thrust = fuelFlow * 45000 Newtons; provides liftCoefficient() using thin airfoil theory
- FighterJet (extends Aircraft, implements SupersonicCapable, MissionPlannable) — overrides dragForce() to include wave drag above Mach 1: waveDrag = 0.1 * (machNo - 1)^2 * refArea * dynamicPressure; implements missionRange() using Breguet range equation: R = (Isp * v * L/D) * ln(initialMass / finalMass); currentPosition() computes x,y coordinates using heading angle and time

Core Computation Task: Simulate a mission where the jet burns from 95% to 40% fuel load. Compute: max range via Breguet, combat radius, heat at Mach 2.2, sonic boom overpressure at 15,000m altitude, and kinetic energy at max speed.

BATCH B

Q1. **Portfolio Risk Analyzer:** Define interfaces EquityAnalyzer and DebtAnalyzer. EquityAnalyzer provides a default method sharpeRatio(double returnRate, double riskFreeRate, double stdDev) = (return - riskFree) / stdDev. DebtAnalyzer provides a default method yieldToMaturity(double faceValue, double couponRate, double price, int years). A MixedPortfolio class implements both, calculates weighted portfolio return, overall Sharpe ratio across both asset classes, and generates a rebalancing recommendation based on target allocation drift > 5%.

Q2. **Multi-Tier Banking & Investment Platform**

Inheritance Structure:

Transactable
└── Investable (extends Transactable)
└── RetirementInvestable (extends Investable)

Auditable
└── RegulatoryCompliant (extends Auditable)

Account (base class)
└── InvestmentAccount (extends Account)
└── PensionFundAccount (extends InvestmentAccount)
implements RetirementInvestable, RegulatoryCompliant

Interface Definitions:

- Transactable — abstract methods: deposit(double), withdraw(double); default method: transactionFee(double amount) → returns 0.2% of amount
- Investable (extends Transactable) — abstract method: allocateFunds(double amount, String asset); default method:

expectedReturn(double principal, double rate, int years) using compound interest: P(1 + r/n)^(nt) compounded quarterly

- RetirementInvestable (extends Investable) — abstract method: calculateMaturityValue(int yearsToRetirement); default method: inflationAdjustedValue(double futureValue, double inflationRate, int years) → FV / (1 + inflation)^years
- Auditable — default method: generateRiskScore(double volatility, int missedPayments) → score = (volatility * 40) + (missedPayments * 15), capped at 100
- RegulatoryCompliant (extends Auditable) — abstract method: submitComplianceReport(); default method: penaltyForNonCompliance(double portfolioValue) → 3.5% of portfolio if risk score > 70

Class Definitions:

- Account — fields: accountNumber, holderName, balance; concrete methods: getBalance(), printStatement()
- InvestmentAccount (extends Account) — adds portfolioValue, riskAppetite; implements deposit() and withdraw() with balance guard
- PensionFundAccount (extends InvestmentAccount, implements RetirementInvestable, RegulatoryCompliant) — must resolve diamond conflict between Transactable.transactionFee() inherited through two interface chains; overrides transactionFee() to return 0.05% for pension accounts; implements calculateMaturityValue() using stepped contribution model (contributions increase 5% YoY); submitComplianceReport() computes total tax liability under Section 80C (deduction up to ₹1.5L), net taxable corpus, and flags if penalty applies

Core Computation Task: For a 30-year-old investing ₹10,000/month until age 60, compute: inflation-adjusted maturity value, tax-exempt corpus, penalty liability if non-compliant, and a year-by-year growth table.

BATCH C

Q1. **Flight Fuel and Emission Calculator:** Create FuelCalculable and EmissionTrackable interfaces. FuelCalculable has a default method estimateFuelBurn(double distanceKm, int passengers) — base burn rate 4.5 L/km + 0.002 L per passenger per km. EmissionTrackable has a default method co2Emissions(double fuelLiters) — jet fuel emits 2.53 kg CO_2/L. An AircraftFlight class implements both, computes total fuel cost at ₹85/L, per-passenger fuel surcharge, carbon offset cost at ₹1,200/tonne CO_2, and net ticket cost breakdown.

Q2. **Hospital Management & Medical Billing System**

Inheritance Structure:

Diagnosable
└── Treatable (extends Diagnosable)
└── SurgicallyTreatable (extends Treatable)

Billable
└── InsuranceClaimable (extends Billable)
└── CorporateClaimable (extends InsuranceClaimable)

Person (base class)
└── Patient (extends Person)
└── SurgicalPatient (extends Patient)
implements SurgicallyTreatable, CorporateClaimable

Interface Definitions:

- Diagnosable — abstract: runDiagnostics(String[] tests); default: diagnosticCost(int numberOfTests) → base ₹800 per test + ₹200 lab processing fee flat
- Treatable (extends Diagnosable) — abstract: prescribeMedication(String condition); default: treatmentDuration(int severity) → days = severity * 3 + 5; default:

medicationCost(int daysOfTreatment, double dailyDrugCost) → total + 12% pharmacy markup

- SurgicallyTreatable (extends Treatable) — abstract: scheduleSurgery(String type, int urgencyLevel); default: surgeryRisk(int age, double bmi, int comorbidities) → risk score = (age * 0.4) + (bmi > 30 ? 20 : 0) + (comorbidities * 15); default: anesthesiaCost(double surgeryHours) → ₹3,500/hour + ₹2,000 flat
- Billable — abstract: generateInvoice(); default: applyGST(double amount) → 5% on medical, 18% on non-medical services; default: lateFee(int daysOverdue) → 1.5% per month compounded
- InsuranceClaimable (extends Billable) — abstract: submitClaim(String policyNumber); default: reimbursementAmount(double totalBill, double coveragePercent, double deductible) → max(0, totalBill * coverage - deductible)
- CorporateClaimable (extends InsuranceClaimable) — default: corporateDiscount(double bill, String tier) → Gold: 20%, Silver: 12%, Standard: 5%; abstract: validateEmployeePolicy(String empId)

Class Definitions:

- Person — fields: name, age, gender, contactInfo; method: bmi(double weight, double height) → weight / height^2
- Patient (extends Person) — adds patientId, admissionDate, roomType; computes roomCharges(int days) — ICU: ₹12,000/day, Private: ₹5,500, General: ₹1,800; implements generateInvoice() summing all charge components
- SurgicalPatient (extends Patient, implements SurgicallyTreatable, CorporateClaimable) — must resolve conflict where applyGST() is reachable from two interface chains (SurgicallyTreatable → Treatable → Diagnosable → ... and CorporateClaimable → InsuranceClaimable → Billable); overrides applyGST() to apply 5% only on surgery and 0% on diagnostics; scheduleSurgery() computes OR booking cost (₹25,000 base + ₹8,000/hour); final

bill computation: diagnostics + medication + surgery + anesthesia + room → GST → corporate discount → insurance reimbursement → out-of-pocket

Core Computation Task: A 55-year-old corporate employee (BMI 32) undergoes a 3-hour surgery (severity 7). Compute: full itemized bill, risk score, corporate Gold-tier discount, insurance reimbursement (80% coverage, ₹50,000 deductible), and final out-of-pocket payment.

BATCH D:

Q1. **Energy Consumption Monitor:** Create interfaces SolarPowered and GridPowered, each with a default method calculateCostPerUnit() — solar returns ₹2.5/unit, grid returns ₹7.0/unit. A HybridHome class implements both, overriding the method to compute a weighted average cost based on percentage usage (e.g., 60% solar, 40% grid). Add an abstract method monthlyBill(double units) that HybridHome implements using the weighted cost, along with carbon footprint calculation (grid emits 0.82 kg CO_2/unit).

Q2. **Smart City Energy Grid Management System**

Inheritance Structure:

PowerGenerable
└── RenewableGenerable (extends PowerGenerable)
└── StorageCapable (extends RenewableGenerable)

GridManageable
└── LoadBalanceable (extends GridManageable)
└── SmartMeteringEnabled (extends LoadBalanceable)

Infrastructure (base class)
└── PowerStation (extends Infrastructure)
└── SmartRenewableGrid (extends PowerStation)
implements StorageCapable, SmartMeteringEnabled

Interface Definitions:

- PowerGenerable — abstract: generatePower(double inputResource); default: transmissionLoss(double generatedKWh, double distanceKm) → loss = generatedKWh * 0.03 * sqrt(distanceKm) (3% base + distance factor); default: carbonCredit(double renewablePercent, double totalKWh) → credits = (renewablePercent/100) * totalKWh * 0.82

- RenewableGenerable (extends PowerGenerable) — abstract: capacityFactor(double actualOutput, double ratedCapacity); default: solarOutput(double panelAreaM2, double irradiance, double efficiency) → panelArea * irradiance * efficiency; default: windOutput(double bladeRadius, double windSpeed, double efficiency) → 0.5 * 1.225 * π * bladeRadius^2 * windSpeed^3 * efficiency
- StorageCapable (extends RenewableGenerable) — abstract: chargeBattery(double excessKWh); default: batteryEfficiency(double storedKWh, double retrievedKWh) → round-trip efficiency %; default: optimalDischargeTime(double storedKWh, double peakDemandKW) → hours of peak coverage = storedKWh / peakDemandKW
- GridManageable — abstract: regulateVoltage(double currentLoad); default: gridFrequency(double activePower, double reactivePower) → power factor = P / sqrt(P^2 + Q^2); default: peakDemandSurcharge(double peakKW, double avgKW) → surcharge applies if peak > 1.3 * avg
- LoadBalanceable (extends GridManageable) — abstract: redistributeLoad(double[] zoneLoads); default: loadVariance(double[] zoneLoads) → statistical variance across zones; default: spilloverRisk(double totalSupply, double totalDemand) → risk ratio = totalDemand / totalSupply
- SmartMeteringEnabled (extends LoadBalanceable) — abstract: readMeterData(String zoneId); default: timeOfUseBilling(double kWh, String timeSlot) → off-peak: ₹4.2, shoulder: ₹6.8, peak: ₹11.5/kWh; default: demandResponseIncentive(double reducedKWh) → savings = reducedKWh * ₹3.5 credited back

Class Definitions:

- Infrastructure — fields: facilityId, location, commissionedYear, operationalCost; method: depreciatedValue(double originalCost, int years) → straight-line at 5%/year; method:

maintenanceCost(double assetValue) → 2.5% of current depreciated value

- PowerStation (extends Infrastructure) — adds installedCapacityMW, gridZonesCovered, currentLoad; implements generatePower() using thermal efficiency model; implements regulateVoltage() using droop control formula: voltageCorrection = (nominalVoltage - currentVoltage) / droop; concrete method: efficiencyRating() = actualOutput / installedCapacity * 100
- SmartRenewableGrid (extends PowerStation, implements StorageCapable, SmartMeteringEnabled) — resolves conflict where transmissionLoss() and carbonCredit() are accessible through both StorageCapable and SmartMeteringEnabled interface chains; overrides transmissionLoss() to apply smart grid correction factor (15% better than default); redistributeLoad() implements a proportional fairness algorithm across zones minimizing variance; chargeBattery() stores only when solar+wind output exceeds demand by >10%; computes 24-hour energy dispatch plan optimizing battery charge/discharge cycles, ToU billing revenue, and demand response savings

Core Computation Task: A smart grid has 50MW solar (18% efficiency, 8 hrs irradiance), 30MW wind (Cp=0.42, avg 9m/s), and 200MWh battery storage. Compute: daily renewable generation, transmission losses, battery cycles, zone-wise ToU billing revenue across peak/off-peak hours, carbon credits earned, and net operating profit after maintenance and depreciation.